

Revealing hidden dynamics from time-series data by ODENet

Pipi Hu^{a,b,1}, Wuyue Yang^{a,1}, Yi Zhu^{a,b}, Liu Hong^{c,*}

^a Yau Mathematical Sciences Center, Tsinghua University, Beijing, 100084, China

^b Yanqi Lake Beijing Institute of Mathematical Sciences and Applications, Beijing 101408, China

^c School of Mathematics, Sun Yat-Sen University, Guangzhou, 510275, China

ARTICLE INFO

Article history:

Received 20 October 2020

Received in revised form 9 February 2022

Accepted 3 April 2022

Available online 7 April 2022

Keywords:

ODENet

Time-series data

Ordinary differential equations

Chemical reactions

ABSTRACT

To derive the hidden dynamics from observed data is one of the fundamental but also challenging problems in many different fields. In this study, we propose a new type of interpretable network called the ordinary differential equation network (ODENet), in which the numerical integration of explicit ordinary differential equations (ODEs) is embedded into the machine learning scheme to build a general framework for revealing the hidden dynamics buried in massive time-series data efficiently and reliably. ODENet takes full advantage of both machine learning algorithms and ODE modeling. On one hand, the embedding of ODEs makes the framework more interpretable benefiting from the mature theories of ODEs. On the other hand, the schemes of machine learning enable data handling, paralleling, and optimization to be easily and efficiently implemented. From classical Lotka-Volterra equations to chaotic Lorenz equations, the ODENet exhibits its remarkable capability in handling time-series data even in the presence of large noise. We further apply the ODENet to real actin aggregation data, which shows an impressive performance as well. These results demonstrate the superiority of ODENet in dealing with noisy data, data with either non-equal spacing or large sampling time steps over other traditional machine learning algorithms.

© 2022 Elsevier Inc. All rights reserved.

1. Introduction

At every moment, massive data has been collected through diverse human activities. And revealing the hidden dynamics from those collected time-series data is one fundamental goal of science. There are many “standard” theories to describe such dynamics, among which differential equations are probably the most successful ones. For example, Newton’s equation $F = m\ddot{x}$ combined with the law of universal gravitation gives a simple and correct picture to explain the complicated motions of planets and the sun and even the existence of Pluto more than half a century before its discovery. However, many new fields, like psychology and social science, still lack rigorous and quantitative theories/equations until now. Therefore, how to construct effective models from a data-driven point of view becomes an interesting topic.

Unfortunately, the time-series data in the record usually contain a lot of missing points and even flaws. They are also highly noisy, with useful signals deeply buried. These facts make analyzing time-series data and extracting useful models or principles hard and tricky. A most famous example is the explanation of planetary orbits in the solar system. Even though wrongly placing the earth at the center, Claudius Ptolemy was still able to explain the motion of planets and the sun by

* Corresponding author.

E-mail addresses: yizhu@mail.tsinghua.edu.cn (Y. Zhu), hongliu@sysu.edu.cn (L. Hong).

¹ These authors have contributed equally to this work.

obscure deferent and epicycle with certain accuracy. Only 1,400 years later, after the landmark works of Copernicus, Kepler, and Newton, a much simpler and more correct picture could be gradually established and accepted. This story highlights the ambiguity and difficulty behind data-driven modeling.

In recent years, the analysis of time-series data has already become a specific subject [1]. Especially in the presence of big data, plenty of machine learning algorithms, like recurrent neural network (RNN) [2], long short-term memory (LSTM) [3], etc., have been widely applied to various fields. RNN, which uses its internal state network to store representations of recent inputs and reuse the output to process the time series, shows a dramatic different “memory” property and network flow structure from other supervised neural networks. LSTM solves the problems of vanishing gradients in RNN by using the so-called “long term” and “short term” memories. Though RNN and LSTM are very successful in applications, their performance in dealing with time-series data collected from physical processes is not very promising [4]. By adding short-cuts to jump over some layers, the ResNet [5] can avoid the problem of vanishing gradients and shows better performances than classical deep learning networks. Those short-cuts can also be treated as some kind of “memory”, which keeps the intrinsic properties unchanged during the learning procedure. Mathematically, the ResNet is analogous to the numerical schemes of ODEs. This interesting connection leads to the so-called continuous view of machine learning, which has been explored a lot from a mathematical point of view [6,7] and also been used for constructing new machine learning algorithms [4,8].

Besides the aforementioned neural networks, there are many other attempts to extract dynamical equations from the time-series data in the past years. For example, Bongard and Schmidt used symbolic regression to find nonlinear differential equations [9–11]. Kutz et al. proposed a framework named “SINDy” by combining regression and sparse identification to reveal nonlinear dynamical systems [12]. To stabilize the performance of sparse identification in the presence of noise, some technical schemes for differential operations were proposed [13,14]. Recently, Dong et al. [15,16] use kernels in CNN which mimic the differential operators to reveal the PDE dynamics from the training data. The Physical Informed Neural Network (PINN) proposed by Raissi et al. leverages the automatic differentiation of neural networks to solve differential equations [17] or reveal differential equations from the data [18]. Recent related works on the forward or inverse problems of differential equations by neural networks refer to [19–24].

Inspired by the great success of modern machine learning algorithms, in the current study we aim to reveal the hidden dynamics from the time-series data without prior knowledge. Different from most previous works that focus on the efficiency and accuracy of predictions of neural networks with little or no physical understanding, we are more interested in deriving the explicit governing equations from the time-series data, which is done under the help of a new type of network, called the ordinary differential equation network (ODENet). By combining the optimization structure of neural networks with symbolic regression, sparse identification, and signal-noise decomposition, ODENet exhibits an outstanding ability in deriving the explicit ODE models for population dynamics modeled by Lotka-Volterra equations in the presence of large noise, strange-attractors of Lorenz equations in the chaotic region, as well as the hidden actin growth dynamics and molecular mechanisms based on real experimental data under distinct conditions. In particular, through these studies, our framework has been proven to be robust, noise-tolerant, immune to unequal time steps of training data, and therefore ODENet is quite suitable for time-series data analysis and data-driven mathematical modeling.

2. The architecture of ODENet

There are two ways to interpret the dynamics behind the time-series data. The usual machine learning algorithms, like LSTM and deep learning, tend to use a vast neural network containing a large number of free parameters to achieve the goal of representing a complex mapping function that best fits the data set. Through iterative training and optimization, the data correlation is transformed into very complicated and thus unexplainable relations among network nodes. In contrast, regression and sparse identification methods adopt an alternative view, which is more concerned about the construction of explicit relations or dynamic differential equations for a globally fitting of the data but only with a few parameters. Each way has its advantages and appropriate applicable regions. Since we are more interested in the physical mechanisms and mathematical models behind the data, the parts of the two ways are combined and realized through the ODENet in the current study.

As a modification of ResNet, the basic structure of ODENet (see Fig. 1) mimics the numerical solvation of ODEs with unspecified parameters to be learned from the data. Through iteratively minimizing the difference between predicted time trajectories and training data measured by a certain loss function, unknown parameters are optimized in such a way that the most suitable ODE model for the given time-series data is explicitly specified.

Data Batching: To be concrete, the learning procedure of the ODENet begins with m randomly selected points from the training data set, $\mathbf{x}_i(t_0)$, for $i = 1, 2, \dots, m$, which will be used as the starting points for integration. For each starting point, its following n successive data are picked as labels. Till now, we have extracted m pieces of time-series data of length $n + 1$, i.e. $\mathbf{x}_1(t_0), \mathbf{x}_1(t_1), \dots, \mathbf{x}_1(t_n)$; $\mathbf{x}_2(t_0), \mathbf{x}_2(t_1), \dots, \mathbf{x}_2(t_n)$; $\dots$; $\mathbf{x}_m(t_0), \mathbf{x}_m(t_1), \dots, \mathbf{x}_m(t_n)$, which constitute the batch shown in Fig. 1b. In general, $\mathbf{x}_i(t_j)$ is a d -dimensional vector, where index i represents the i^{th} piece and index j represents the $(j + 1)^{\text{th}}$ point in the time-series data counting from the starting one. Note $t_i - t_{i-1}$ may not necessarily be the same for different $i \in \{0, 1, 2, \dots, n\}$. However, as large Δt will make the problem stiff while small Δt may be unable to provide sufficient information for the dynamics, the time steps must be carefully adjusted in order to keep a well balance between model accuracy and stiffness.

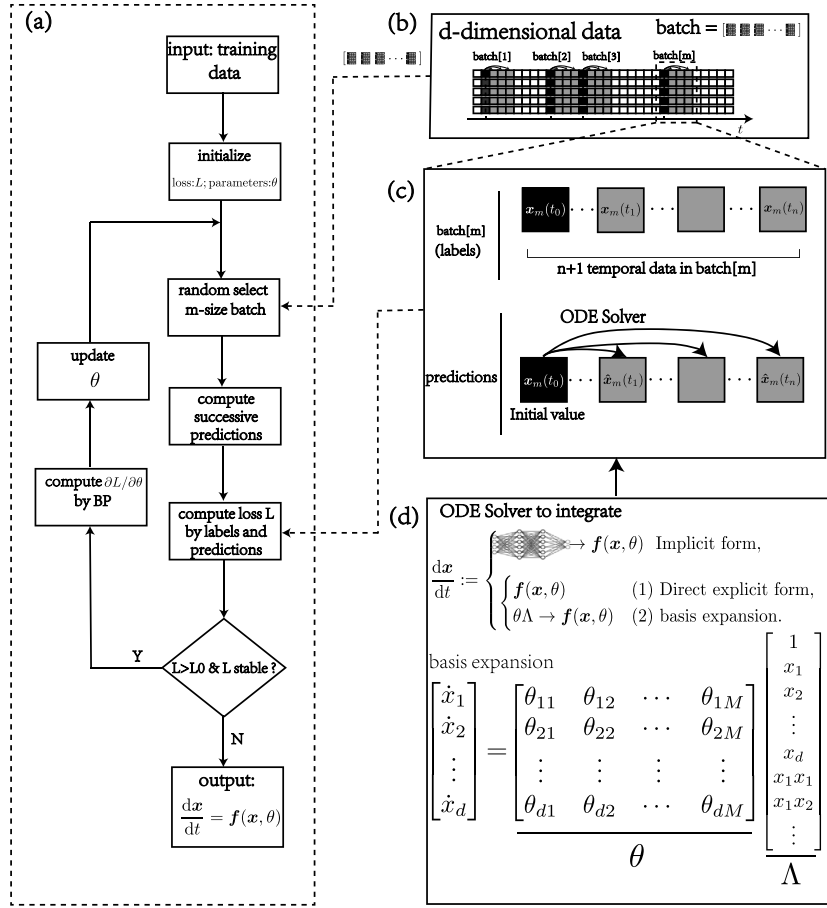


Fig. 1. The architecture of ODENet with key steps highlighted in the right columns. (a) shows the basic flowchart of the training algorithm, (b) exhibits the structure of training data, and (c) rooms in to show the data structure of one training piece in one batch, and (d) gives the mathematical schemes of ODEs.

ODE Dynamics: Next, referring to each piece of data in the batch, we suppose they satisfy the following initial-value problem of a system of autonomous ordinary differential equations,

$$\begin{aligned} \frac{d\mathbf{x}}{dt} &= \mathbf{f}(\mathbf{x}; \boldsymbol{\theta}), \\ \mathbf{x}(t_0) &= (x_1(t_0), x_2(t_0), \dots, x_d(t_0))^T. \end{aligned} \quad (1)$$

This is the key assumption of our ODENet. Here, the starting point $\mathbf{x}(t_0)$ at t_0 is taken as the initial value. $\boldsymbol{\theta}$ stands for the unspecified parameters, which determine the explicit form of the right-hand side terms, and will be addressed later. Next, above ODEs will be solved numerically by mutual ODE solvers, like the Runge-Kutta method, whose numerical solutions at $t_1, \dots, t_n$ are denoted as $\hat{\mathbf{x}}(t_1), \hat{\mathbf{x}}(t_2), \dots, \hat{\mathbf{x}}(t_n)$ respectively, in order to distinguish from the labels $\mathbf{x}(t_1), \mathbf{x}(t_2), \dots, \mathbf{x}(t_n)$.

Therefore, we actually adopt an integral method to try to predict the ODE dynamics starting from $\mathbf{x}(t_0)$ at t_0 . Unlike most previous regression based methods, e.g. SINDy [12], our approach minimizes the deviations from the training data $\mathbf{x}(t_1), \mathbf{x}(t_2), \dots, \mathbf{x}(t_n)$ at time $t_1, t_2, \dots, t_n$ instead of the derivatives. We will come back to this point soon.

Approximation of Unknown Functions: Before turning to the description of the loss function and optimization scheme, we emphasis here the structure of the right-hand side terms $\mathbf{f}(\mathbf{x}; \boldsymbol{\theta}) = (f_1, f_2, \dots, f_d)^T$, whose form completely determines the ODE dynamics in (1) and has to be found out from the training data.

There are two different ways – the implicit and the explicit. The usual neural networks, especially deep neural networks, which show a great ability to approximate very complicated mapping functions, in reality, belong to the former since they often contain multiple layers, tremendous nodes, and connections with massive adjustable free parameters. In most cases, they are “black boxes” to us. The implicit approach has been adopted by several groups previously and applied to MINIST as an example [4,8].

In the current study, as we are more interested in the dynamics and mechanism behind the time-series data, we hope to derive the ODE model in an explicit way. To achieve such a goal, the right-hand side terms $\mathbf{f}(\mathbf{x}; \boldsymbol{\theta})$ are expanded through

basis functions with corresponding coefficients θ . Although the selection of proper basis function is quite tricky and problem dependent, polynomials are among the most often used ones in practice. For the d -dimensional vector $\mathbf{x} = (x_1, x_2, \dots, x_d)^T$, the p^{th} -order complete polynomials

$$\{1, x_1, x_2, \dots, x_d, x_1x_1, x_1x_2, \dots, x_d^p\}$$

have $M = \binom{p+d}{p}$ terms. So that we have

$$\mathbf{f}(\mathbf{x}; \theta) = \theta \Lambda, \quad (2)$$

where $\Lambda = (1, x_1, x_2, \dots, x_d, x_1x_1, x_1x_2, \dots, x_d^p)^T$ is the complete set of p^{th} -order polynomial basis with coefficients $\theta = (\theta_{ij})_{d \times M}$.

Loss Function: By tuning free parameters θ , which specify the concrete form of ODEs, the loss function

$$\mathcal{L}(\theta; \lambda) = \|\mathbf{x} - \hat{\mathbf{x}}\|_1 + \mu \|\theta\|_1 \quad (3)$$

is expected to be minimized. Here the first term characterizes the difference between the training data and predictions, while the second term represents the sparsity requirement with $\mu > 0$ as a hyper parameter. According to Occam's Razor, "plurality should not be posited without necessity." So it is expected that the components of θ should be as many zeros as possible, which thus corresponds to the simplest model and also the smallest $L1$ norm. Here we may encounter the $L1$ optimization problem. The autograd adopted in PyTorch uses subderivatives for calculating gradients of $L1$ norm at the non-differentiable points. We refer readers to alternative new algorithms, such as the Split Bregman Method [25], Coordinate Descent [26], Proximal Gradient Method [27] and etc., which may offer a better performance on optimizing $L1$ norm than the subgradients.

Besides μ , an additional threshold parameter $\gamma > 0$ is adopted to accelerate the learning of sparse models. Once a component of θ is smaller than γ , it will be forced to be zero in the remaining learning process. As a consequence, γ should be carefully tuned to make a good balance between model simplicity and fitting accuracy. Generally speaking, within a certain region, the larger the threshold is, the faster the loss function will converge. However, once the threshold becomes too large, the learning process has a very high risk of failing due to too many abandoned terms.

Parameter Optimization: The searching of the best θ is a global optimization problem, which can be done via mature learning algorithms in the neural network, for example, gradient descent or stochastic gradient descent such as the Adam method [28] combined with backward propagation algorithms [29]. Especially in PyTorch, this can be simply done by an integrated autograd and optimization library. Once the forward flow is constructed, the backward flow will be automatically built by Pytorch [30]. It should be noted that autograd is an automatic differentiation method, whose computational graph will become too large and make the problem computationally very expensive once we want to simulate the system for a long time. To avoid this difficulty, we have to look into the problem of stiffness, which is quite often encountered in numerical simulations. Finally, repeat the above procedure iteratively until the loss function does not decay efficiently anymore or less than a threshold, we finish the learning procedure of ODENet, which is summarized through pseudocodes as below.

Algorithm 1: Pesudocode of ODENet (PyTorch).

Input: time-series data $\mathbf{x}(0), \mathbf{x}(t_1), \dots, \mathbf{x}(t_n)$, where $\mathbf{x} = (x_1, x_2, \dots, x_d)^T$.
Output: d-dimensional first order ODEs $\frac{dx}{dt} = \mathbf{f}(\mathbf{x}, \theta)$.
 initialize parameters θ ;
 initialize hyperparameters: *threshold_L*, γ , m, n ;
while $L > \text{threshold_L}$ **do**
 // Construct one batch
 select n -length intervals from m random positions for a batch;
 $\text{batch} \leftarrow [[x_0^1, x_1^1, \dots, x_n^1], [x_0^2, x_1^2, \dots, x_n^2], \dots, [x_0^m, x_1^m, \dots, x_n^m]]$;
 $\text{batch_t} \leftarrow [[t_0^1, t_1^1, \dots, t_n^1], [t_0^2, t_1^2, \dots, t_n^2], \dots, [t_0^m, t_1^m, \dots, t_n^m]]$;
 $\text{batch_init} \leftarrow \text{batch}[:, 1]$;
 $\text{batch_label} \leftarrow \text{batch}[:, 2 :]$;
 $\text{batch_size} \leftarrow m$;
 $L \leftarrow 0$;
 for $i \leftarrow 1$ **to** batch_size **do**
 // Compute the predictions based on batch_init $\text{pred}[i, :] \leftarrow \text{ODESolve}(\text{batch_init}[i], \text{batch_t}[i, :], \theta)$;
 // The θ matrix should be sparse matrix
 $L \leftarrow L + \|\text{Abs}(\text{pred}[i, :] - \text{batch_label}[i, :])\|_1 + \mu \|\theta\|_1$;
 end
 $\theta.\text{grad} \leftarrow \frac{\partial L}{\partial \theta}$ by BP algorithm;
 Update parameters θ by the Adam method;
 // Set θ_{ij} which is small enough to be zero
 $\theta[\theta < \gamma] \leftarrow 0$;
end

Stiffness Problem: It is noted that we put no constraint during the step of data selection. On one hand, this provides a great facility in dealing with real data; on the other hand, it also leaves us in danger of facing stiffness problems from time to time during numerical simulations, which becomes especially outstanding when linear combinations of high-order polynomials are taken as the right-hand side terms of an ODE system.

According to the general knowledge of ODE numerics, we suggest the following ways for dealing with the stiffness problem. First, divide long-time trajectories of the training data set into many short pieces. And thus each piece will contain only a few time steps, which could be easily solved by classical gradient methods implemented in PyTorch. Second, towards the ODE solver, we suggest using self-adaptive and/or implicit ODE solvers, which were designed for solving stiff cases for the best [31,32]. In this work, `dopri5` – an explicit scheme combined 4th-order and 5th-order Runge-Kutta methods due to Dormand & Prince with step-size control and dense output, was used as our ODE solver in all experiments presented [33].

Third, instead of arbitrarily setting parameters (coefficients of the polynomial basis) through random number generators, regression methods can be introduced to make a reasonable initial guess on the parameters [12]. Last, for recent advances in the direction of multiscale modeling methods combined with ODE or PDE-based neural networks, which may provide an alternative solution to the stiffness problems, see e.g. Ref. [34,35].

Integral v.s. Differential: One significant feature of the ODENet from previous regression based methods like SINDy [12] is that our approach is based on an integration of explicit ODEs along the time trajectory, while theirs [9,10,12] are all based on differentiation between neighboring points. To be concrete, in our approach the data points on a time trajectory are predicted based on an integral solution of the ODE model, i.e. $\mathbf{x}(t) = \int_{t_0}^{t_n} \mathbf{f}(\mathbf{x}(\tau); \boldsymbol{\theta}) d\tau + \mathbf{x}(t_0)$. The loss function is designed to minimize the difference between the predicted $\mathbf{x}(t)$ and real ones. In contrast, according to the methods reported in Refs. [9,10,12], what they actually tried to minimize is the difference between the function $\mathbf{f}(\mathbf{x}(\tau); \boldsymbol{\theta})$ on the right-hand side of (1) and the time derivative $\dot{\mathbf{x}}(t) = \frac{\mathbf{x}(t) - \mathbf{x}(\tau)}{t - \tau}$ calculated from the training data. Clearly, the latter is a kind of differentiation methods.

The differentiation methods are simple and straightforward, easy for implementation, numerically fast and efficient, and suitable for high-dimensional complicated dynamics. In comparison, the integral methods enjoy advantages like being more stable against faults and flaws in data, more noise-tolerant, able to endure data with large time steps, etc.

Noise: As we have discussed in the introduction, real data may have noise, flaws and faults. To deal with this issue, we take a brutal method here by incorporating the strength of noise as learning parameters too. Suppose there is a finite time series $\mathbf{y}(t) \in \mathbb{R}^d$ with noise $\mathbf{e}(t) = \epsilon \|\mathbf{y}\|_\infty \boldsymbol{\eta}$, where ϵ denotes the noise strength, $\|\mathbf{y}\|_\infty$ is the maximal value in $\mathbf{y}$, and $\boldsymbol{\eta} \sim \mathcal{N}(0, 1)$ are d -dimensional normally distributed random variables. ODENet is proposed to extract the following autonomous system of ODEs

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}; \boldsymbol{\theta}), \quad \mathbf{x}(t) = \mathbf{y}(t) - \mathbf{e}(t) \in \mathbb{R}^d. \quad (4)$$

And we can apply the same process as before to extract the governing dynamics from the data. Apparently, at this time, the loss function depends on $\mathbf{e}(t)$ too, i.e. $L = L(\boldsymbol{\theta}, \mathbf{e}(t); \mu) = \|\hat{\mathbf{x}} + \hat{\mathbf{e}} - \mathbf{y}\|_1 + \mu \|\boldsymbol{\theta}\|_1$, where $\hat{\mathbf{x}}$ and $\hat{\mathbf{e}}$ denote the output of ODENet and learned noise respectively. Although directly incorporating noise as learning parameters may cause a big increment in the computational cost, it allows the treatment of large noise or color noise in principle. If the noise is relatively small, it is more appropriate to try to learn the original data directly and then check its robustness against noise. Interested readers may refer to Refs. [13,14] for alternative solutions.

3. Numerical experiments

In this section, we are going to apply the ODENet to the study of Lotka-Volterra equations in diverse parameter regimes and Lorenz equations in chaotic regimes. Through these examples, the power and advantage of ODENet could be demonstrated.

3.1. Lotka-Volterra equations with and without large noise

The Lotka-Volterra (LV) equations, also known as predator-prey equations, were first introduced by Lotka [36] and Volterra [37] in the 1920s to describe the population dynamics of preys interacting with predators in ecological systems. LV equations have been widely applied to ecological balance [38], environmental protection [39], disease prevention and control [40], etc. A very general form of LV equations including species growth and death, intraspecies and interspecies competition reads

$$\begin{aligned} \frac{dx_1}{dt} &= C_{11}x_1 + C_{12}x_1x_2 + C_{13}x_1^2, \\ \frac{dx_2}{dt} &= C_{21}x_2 + C_{22}x_1x_2 + C_{23}x_2^2. \end{aligned} \quad (5)$$

It's easy to see above equations have four fixed points, i.e. $(0, 0)$, $(-\frac{C_{11}}{C_{13}}, 0)$, $(0, -\frac{C_{21}}{C_{23}})$ and $(-\frac{C_{12} \cdot C_{21} - C_{11} \cdot C_{23}}{C_{12} \cdot C_{22} - C_{13} \cdot C_{23}}, -\frac{C_{11} \cdot C_{22} - C_{13} \cdot C_{21}}{C_{12} \cdot C_{22} - C_{13} \cdot C_{23}})$ under the condition $C_{12} \times C_{22} \neq C_{13} \times C_{23}$, $C_{13} \neq 0$ and $C_{23} \neq 0$. And the dynamic behaviors of LV equations around these fixed points (x_1^*, x_2^*) are fully specified by the Jacobian matrix (its eigenvalues to be exact)

$$J = \begin{bmatrix} C_{11} + C_{12}x_2 + 2C_{13}x_1 & C_{12}x_1 \\ C_{22}x_2 & C_{21} + C_{22}x_1 + 2C_{23}x_2 \end{bmatrix}_{(x_1^*, x_2^*)}, \quad (6)$$

which can be roughly classified into three basic types – the extinction of one species (over damped), or the evolution to an equilibrated coexistence (spiral), or to a continuing oscillation (limit cycle) [41].

Based on the above analysis, the ODENet is applied to learn the dynamics of the LV model (5) within different coefficient regimes, see Fig. 2. As we do not want to introduce any prior knowledge, the right-hand side terms of the ODE system are expanded through a complete polynomial basis. Up to the second order, we have twelve free parameters to learn. The detailed setup can be found in Box 1.

Box 1: Lotka-Volterra models

Goal: Find the correct Lotka-Volterra (LV) model from the given time-series data.

Data: Simulated time trajectories of LV equations representing different types of ODE dynamics in phase space, combined with either small (1%) or large (10%) white noise (with respect to the largest amplitude of data).

Setup: Complete polynomials up to the second order $\Lambda = \{1, x_1, x_2, x_1^2, x_1x_2, x_2^2\}$ with twelve adjustable coefficients $\theta = (\theta_{ij})_{2 \times 6}$ are adopted to approximate function $\mathbf{f}(\mathbf{x})$. For large noise, the noise term $\mathbf{e}(t)$ in (4) is added as learning parameters too.

Learning: Optimize parameters θ and $\mathbf{e}$ following the procedure of Algorithm 1. Parameters θ_{ij} less than the threshold γ is set as zero to remove model redundancy. Regularization parameter μ is decreased from 10^{-3} to 10^{-5} , while threshold γ is increased from 10^{-4} to 10^{-3} with iterations.

Results: The correct form of LV models is reproduced with most terms as zeros. The coefficients of the remaining terms are close to their expected values, with the maximal relative errors less than 6%. The distribution of noise is almost correctly predicted.

As summarized in Table 1, our ODENet shows a competitive performance with the state-of-the-art methods, e.g. SINDy [12]. In fact, for all three typical LV dynamics, all zero terms in the LV model in (5) have been correctly picked out by the ODENet through sparse identification. Furthermore, the maximal relative errors between the learned ones and their true values of the remaining nonzero coefficients are less than 6%. In contrast, SINDy fails to identify the redundant terms in the spiral and limit-cycle cases, whose performance becomes even worse as the data sampling time steps are increased to 0.01.

Even in the presence of large noise, for example in this case up to 10% white noise with respect to the maximal signal value is added to the data (see Fig. 3), our ODENet still shows an astonishing ability in finding out the correct governing equations and revealing the hidden deterministic trajectories which are deeply buried inside noise-spoiled data. The learned noise correctly fits into a Gaussian distribution as expected, though rare events with large displacements are overestimated in the current case, as pointed out in Fig. 3c. Further studies show that the deviation from the standard Gaussian distribution disappears as the noise level is lowered (data not shown). Most importantly, in ODENet, no extra unwanted coefficient will be included in the model as a consequence of sparse identification, even for the flawed and noisy data. This fact is clearly stated through the zero values of C_{13} and C_{23} in the fifth row of Table 1 for LV equations with large noise.

Compared to difference-based methods, the integration-based methods are more tolerant to large time steps in sampling data, as we have claimed. By gradually increasing the sampling time steps of the training data set in the limit-cycle case of LV equations, it is expected that revealing the ODE dynamics from the sample data becomes harder and harder, as less information about the system is included. So that it is not astonishing to see that results of SINDy become untrustable when $\Delta t \geq 0.1$. However, our ODENet still works quite well and shows high accuracy in revealing the dynamics even when $\Delta t = 0.5$ (see Table 2).

3.2. Lorenz equations in chaotic regimes

In the 1960s, American meteorologist Lorenz proposed a simple mathematical system constituted by three ordinary differential equations,

$$\begin{cases} \frac{dx_1}{dt} = C_{11}x_1 + C_{12}x_2, \\ \frac{dx_2}{dt} = C_{21}x_1 + C_{22}x_2 + C_{23}x_1x_3, \\ \frac{dx_3}{dt} = C_{31}x_3 + C_{32}x_1x_2, \end{cases} \quad (7)$$

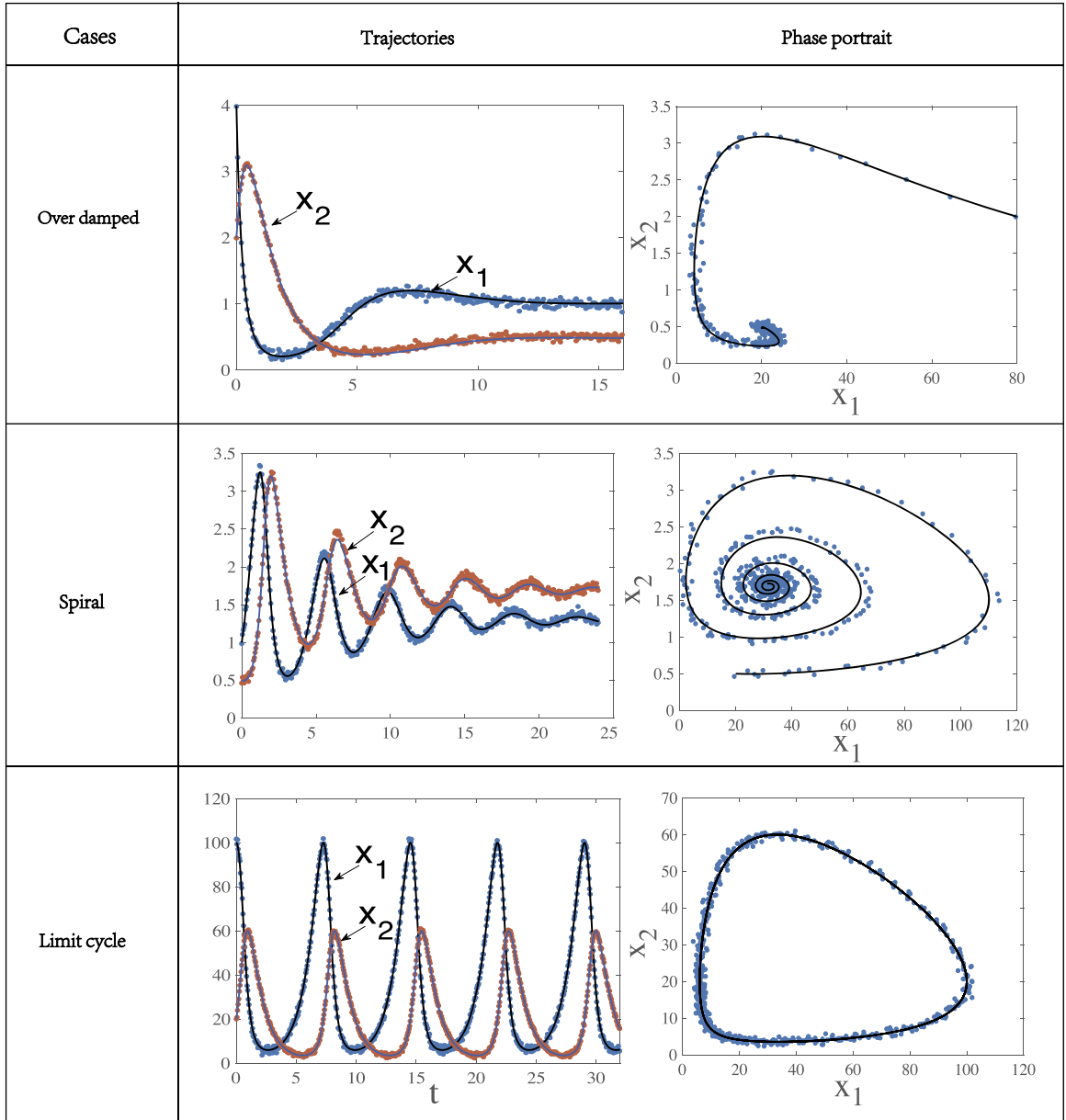


Fig. 2. The accuracy of ODENet predictions in both time domains (left column) and phase space (right column) in comparison with exact solutions of LV equations. 1% white noise is added.

for describing atmospheric turbulence [42]. Lorenz equations became very famous for its chaotic solutions. For a typical parameter combination $C_{11} = -C_{12} = -10$, $C_{21} = 28$, $C_{31} = -8/3$, $C_{22} = C_{23} = -C_{32} = -1$, the Lorenz system has three equilibrium points, i.e., $(0, 0, 0)$, $(6\sqrt{2}, 6\sqrt{2}, 27)$ and $(-6\sqrt{2}, -6\sqrt{2}, 27)$. Numerical simulation shows that typical trajectories of (7) follow a strange attractor in a butterfly shape in the phase space, which first makes a few loops around $(6\sqrt{2}, 6\sqrt{2}, 27)$, then jumps to loops around $(-6\sqrt{2}, -6\sqrt{2}, 27)$, and then come back to the point $(6\sqrt{2}, 6\sqrt{2}, 27)$, again and again (see Fig. 4). In this case, solutions of the Lorenz equations are sensitive to disturbance in the initial conditions, which is widely known as the “butterfly effect” in the literature. Therefore, to catch the charming butterfly from chaotic data is an attractive task, which makes the Lorenz system as a benchmark problem for testing the accuracy of numerical schemes as well as the performance of machine learning algorithms.

To increase the learning difficulty, we introduced white noise with a magnitude up to 0.5% of the maximal signal data, though no disturbance is included in the initial values. According to the results summarized in Table 3 and Fig. 4, ODENet

Table 1

Comparison of ODENet and SINDy on model coefficients for three typical dynamics of LV equations. For comparison, different data sampling time steps are adopted. For ODENet and SINDy#1, $\Delta t = 0.01$, and for SINDy#2, $\Delta t = 0.001$. Instead of 12 coefficients for a 2-order complete polynomial basis, only 6 coefficients correspond to those in (5) are listed for simplicity. No redundant coefficients have been learned by ODENet, in contrast to SINDy. The symbol “—” indicates the failure in deriving a reasonable model from the given data set. In this study, SINDy has been performed under different hyper-parameters for 50 independent runs. The best result is picked out and shown in the table.

LV equations		Parameters					
		C_{11}	C_{12}	C_{13}	C_{21}	C_{22}	C_{23}
Over damped 1% noise	Model	1.5	-1	-1	-1	1	0
	ODENet	1.49	-9.87×10^{-1}	-9.94×10^{-1}	-1.00	9.95×10^{-1}	0
	SINDy#1	—	—	—	—	—	—
	SINDy#2	1.50	-1.00	-1.00	-1.00	1.00	0
	Model	2	-1.1	-0.1	-1	-0.1	0.9
Spiral 1% noise	ODENet	1.99	-1.11	-9.91×10^{-2}	-9.87×10^{-1}	-1.06×10^{-1}	9.02×10^{-1}
	SINDy#1	—	—	—	—	—	—
	SINDy#2	2.18	-8.54×10^{-1}	-1.04×10^{-1}	-9.89×10^{-1}	-1.01×10^{-1}	8.65×10^{-1}
	Model	1	-0.05	0	-1	0.03	0
	ODENet	9.97×10^{-1}	-4.98×10^{-2}	0	-1.00	2.99×10^{-2}	0
Limit cycle 1% noise	SINDy#1	1.00	-4.98×10^{-2}	0	-9.98×10^{-1}	3.01×10^{-2}	0
	SINDy#2	1.01	-4.99×10^{-2}	0	-9.95×10^{-1}	3.00×10^{-2}	0
	Model	1	-0.05	0	-1	0.03	0
Limit cycle 10% noise	ODENet	9.69×10^{-1}	-4.89×10^{-2}	0	-9.82×10^{-1}	2.96×10^{-2}	0
	SINDy#1	9.83×10^{-1}	-5.04×10^{-2}	0	-1.00	3.02×10^{-2}	0
	SINDy#2	1.00	-4.99×10^{-2}	0	-9.97×10^{-1}	3.00×10^{-2}	0
	Model	1	-0.05	0	-1	0.03	0

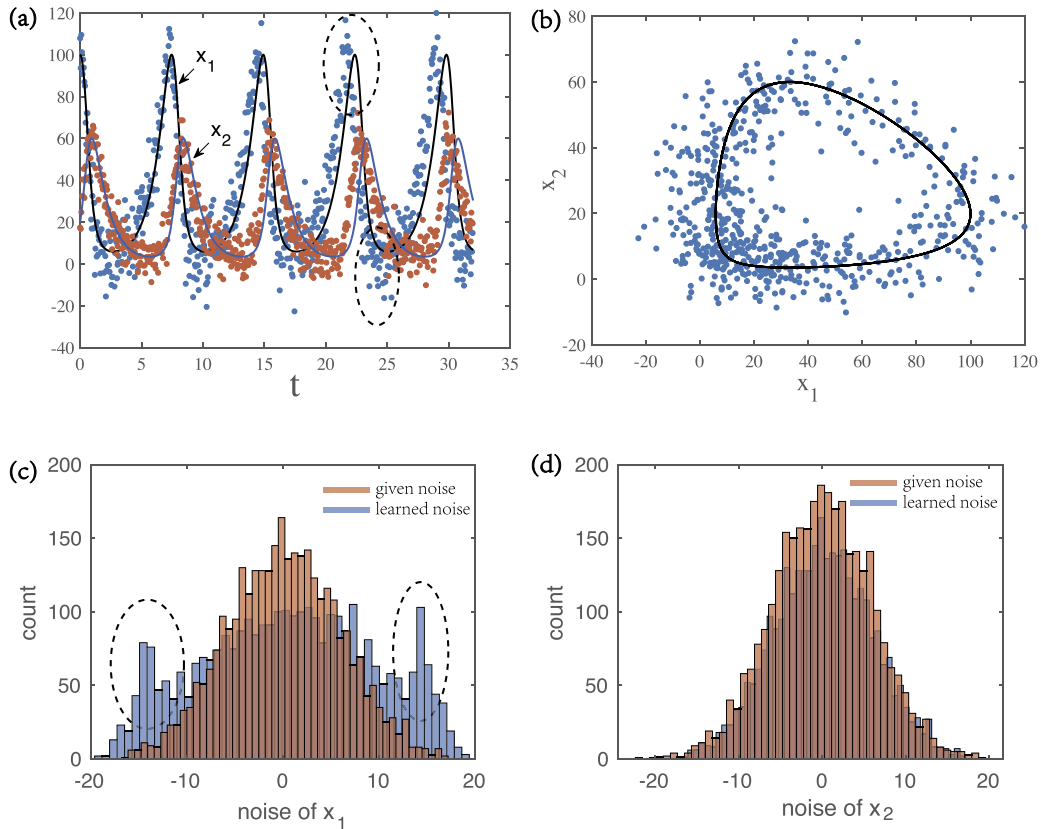


Fig. 3. (a–b) Predictions of ODENet on LV equations in the presence of large external noise (up to 10% of the highest magnitude of the data). Distributions of learned noise in (c) x_1 and (d) x_2 are compared to the real ones. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

Table 2

Influence of data sampling time step on the accuracy of ODENet and SINDy. 1% white noise is added to the limit cycle case of LV equations. Again, redundant coefficients besides the six ones in the given model (5) are omitted.

Limit cycle		Parameters					
		C_{11}	C_{12}	C_{13}	C_{21}	C_{22}	C_{23}
Model		1	-0.05	0	-1	0.03	0
Methods	Δt						
ODENet	0.001	9.93×10^{-1}	-5.00×10^{-2}	0	-1.00	3.00×10^{-2}	0
SINDy	0.001	1.01	-4.99×10^{-2}	0	-9.95×10^{-1}	3.00×10^{-2}	0
ODENet	0.01	9.97×10^{-1}	-4.98×10^{-2}	0	-1.00	2.99×10^{-2}	0
SINDy	0.01	1.00	-4.98×10^{-2}	0	-9.98×10^{-1}	3.01×10^{-2}	0
ODENet	0.1	9.82×10^{-1}	-4.94×10^{-2}	0	-1.00	3.01×10^{-2}	0
SINDy	0.1	5.87×10^{-1}	-5.55×10^{-2}	0	0	1.83×10^{-1}	0
ODENet	0.5	9.90×10^{-1}	-4.99×10^{-2}	0	-1.02	3.08×10^{-2}	0
SINDy	0.5	—	—	—	—	—	—

Table 3

Comparison on the accuracy of ODENet and SINDy for Lorenz equations. For simplicity, only 7 non-zero parameters corresponding to the true model in (7) are listed, while the rest 23 parameters are all zeros for ODENet. There is an extra term -5.115 in the third equation for SINDy.

Lorenz		Parameters					
		C_{11}	C_{12}	C_{21}	C_{22}	C_{23}	C_{31}
Model		-10	10	28	-1	-1	-8/3
ODENet		-9.989	9.982	28.02	-1.008	-1.000	-2.667
SINDy		-9.899	9.976	26.72	-3.965×10^{-1}	-9.747×10^{-1}	-2.447

correctly reproduces the Lorenz attractor, and only a small fraction of trajectories are mispredicted, which is inevitable in the study of chaos.

Box 2: Strange attractors of Lorenz equations

Goal: Predict the strange attractors of Lorenz equations.

Data: Time trajectories of Lorenz equations in the chaotic regime with 0.5% white noise (with respect to the highest amplitude of data) added.

Setup: Complete polynomials up to the second order $\Lambda = \{1, x_1, x_2, x_3, x_1^2, x_1x_2, x_1x_3, x_2^2, x_2x_3, x_3^2\}$ with thirty adjustable coefficients $\theta = (\theta_{ij})_{3 \times 10}$ are adopted to approximate function $\mathbf{f}(\mathbf{x})$. As the orbits are very sensitive to initial values and model coefficients, they are divided into many small pieces to minimize predictive errors.

Learning: Parameters θ are optimized according to the procedure of Algorithm 1. Sparsity requirement is taken. The regularization factor μ is set as a decreasing hyper-parameter from 10^{-4} to 10^{-8} with iterations. The threshold γ is an increasing parameter from 10^{-4} to 10^{-3} to remove redundant terms.

Results: The correct form of Lorenz equations are reproduced with the maximal relative errors of coefficients less than 1%. The strange attractors are correctly predicted even for a long time.

For simplicity, in the above two examples, only the second-order complete polynomials have been used to construct the initial model for the ODENet. And luckily the classical LV and Lorenz equations all fall into this category. If higher-order basis functions are adopted, the correct dynamics, in general, could also be revealed, but at a price of larger computational costs and higher risks of facing stiffness problems. This issue has been tested on the LV equations with respect to third-order and fourth-order complete polynomial basis, in which all coefficients for high-order polynomials (above 2) have been correctly eliminated by choosing suitable threshold parameters (data not shown). And in the case of Lorenz equations, the ODENet works in second-order to fifth-order complete polynomial bases but failed to find the correct answer when the right-hand side terms were expanded higher than fifth-order polynomials, probably due to too many redundant terms. We call the attention of readers to the trade-off between the accuracy and efficiency of our method.

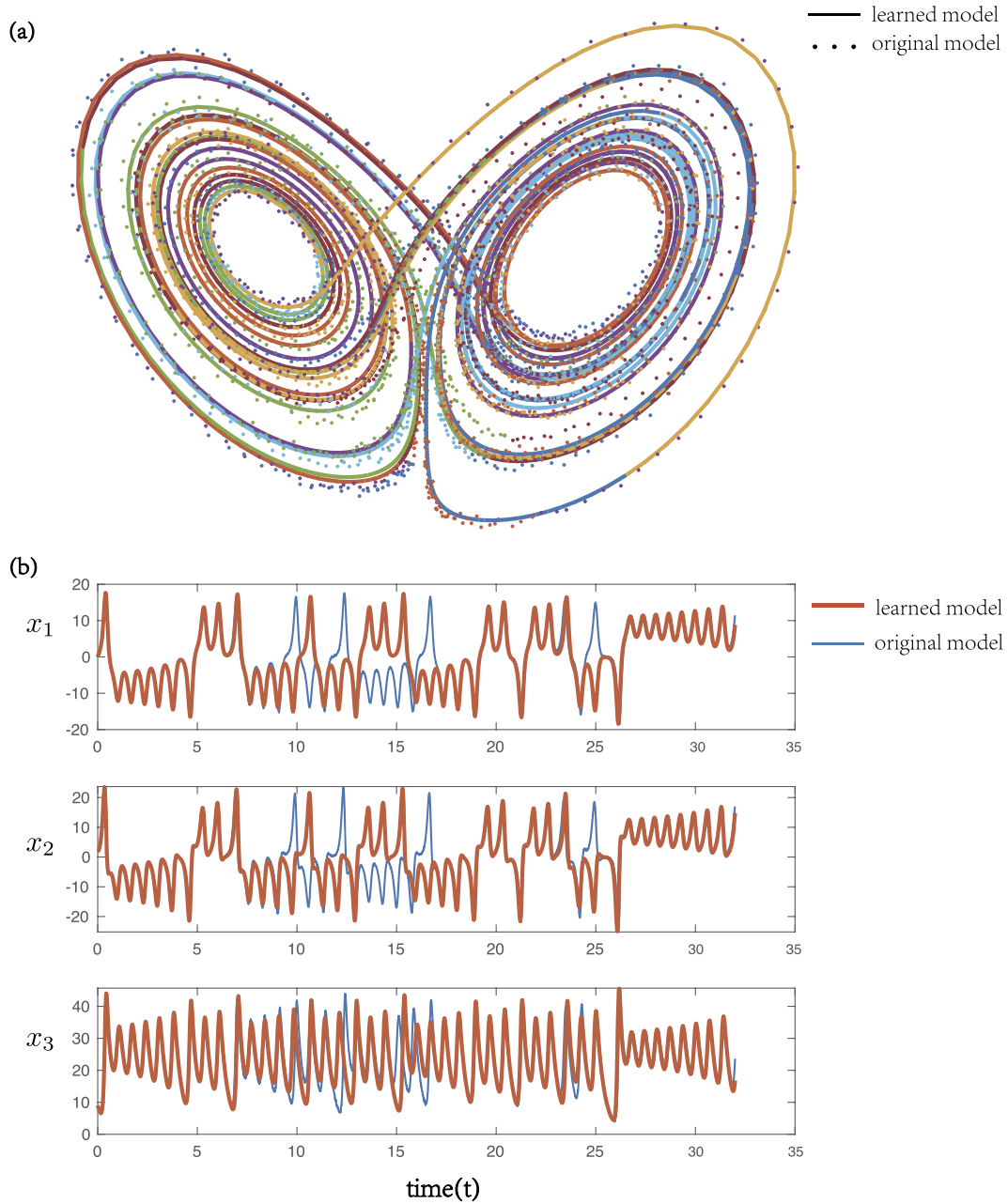


Fig. 4. Predictions of ODENet on Lorenz equations in the chaotic regime.

3.3. The generalization ability of the ODENet

The generalization ability of the ODENet relies on the assumption that there is hidden dynamics characterized by ODEs behind the data. Once the dynamics is correctly revealed by the ODENet from the training data, the explicit ODE model will guarantee a good generalization on the test data.

To quantify the generalization ability of the ODENet, the following numerical experiment is designed. We train the ODENet for the LV model in the spiral regime and use 80% of the total data points as the training set and the rest 20% as the test set. The hyper-parameters and other setups are set the same as those in Section 3.1.

The losses in the training set and the test set are calculated based on the following scheme. (1) Reveal the hidden dynamics from the training set by using the ODENet. (2) Compute the predictions at N sequential time points in the training set and get the loss $L_{train} = \frac{1}{N} \sum_{i=1}^N |x_i^{pred} - x_i^{train}|$. (3) Compute the predictions at N sequential time points in the

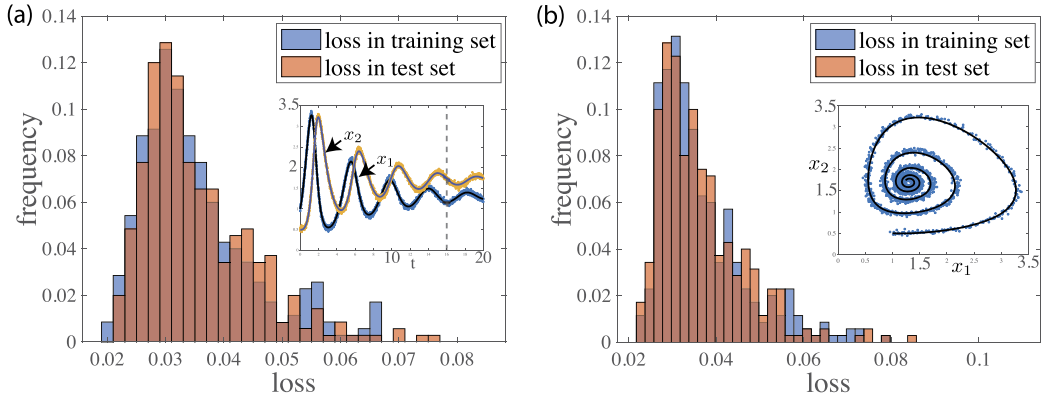


Fig. 5. The generalization ability of the ODENet for the LV model in the spiral regime. The distributions of losses in the training set and test set with either 20 (a) or 40 (b) predicted time points are exhibited. The insets in (a-b) show the learned time series and trajectories of the LV model.

test set and get the loss $L_{test} = \frac{1}{N} \sum_{i=1}^N |x_i^{pred} - x_i^{test}|$. (4) Randomly select two new time points as the starting points and repeat the last two steps.

In the current case, a collection of 350×2 losses are computed for the training set and test set respectively. As illustrated by the distributions in Fig. 5, it can be concluded that there is not much difference in the losses between the training set and test set. This shows the good generalization ability of our ODENet.

4. Application to the kinetics of actin aggregation

Actin aggregation into microfilaments is responsible for the contraction of muscle cells and the motility of other cells. In the 1960s, the first analytical molecular model was proposed by Oosawa et al. [43], which stated the mechanism of actin aggregation includes three basic steps – primary nucleation, elongation, and fragmentation. Primary nucleation is an initialization step to generate new growth seeds through a self-organization process. Then small seeds grow into long actin filaments by elongation, meaning monomeric actions are added to the filament end sequentially. The actin aggregation could be dramatically sped up by fragmentation, through which massive new seeds are generated by breaking long filaments into two shorter pieces without involving primary nucleation. It should be mentioned, besides those forward processes for actin growth, the corresponding inverse processes, like monomer dissociation and fibril annealing, may also make a non-negligible contribution to maintaining the equilibrium distribution of actin filaments. Based on the theory of chemical kinetics, the above picture can be explicitly transformed into a mathematical language of ordinary differential equations, which establishes a direct connection between experimental data and molecular mechanisms of actin growth.

Experimental data: In this study, we re-examine the classical experiments done by Wegner et al. [44], which studied the phenomenon of actin aggregation under two distinct conditions. One is varied concentrations of monomeric actins for $m_{tot} = 7.4, 9.6, 12.4, 14.2, 16.2, 18.4, 20.5 \mu M$ incubated with $40mM$ KCl (Fig. 6a), the other is $m_{tot} = 6.7, 8.5, 11.5, 14.9, 17.3, 20.3, 22.9 \mu M$ actins incubated with $0.6mM$ $MgCl_2$ and $0.5mM$ EGTA (Fig. 6b). The red circles in Fig. 6a and 6b indicate the mass concentration of actin filaments $M(t)$ which is measured through the ThT fluorescence intensity under seven different concentrations of monomeric actins. Clearly, the data are not equally spaced, and the sampling time step Δt is not very small.

To explore the influence of pre-knowledge (or physical insight) on machine learning-based modeling, here we adopt two different setups – one is purely data-driven, the other is physical-based, which, as we will see, leads to models in distinct forms, but all fit the data quite well.

Purely data-driven model: Firstly, we study the pure data-driven modeling without including any pre-knowledge. To account for the concentration dependence, an additional variable – the actin monomer concentration $m(t) = m_{tot} - M(t)$ is introduced besides $M(t)$. As a consequence, we need to learn two ordinary differential equations from the data, i.e.

$$\begin{aligned} \frac{dM}{dt} &= \alpha_0 + \alpha_1 M + \alpha_2 m + \alpha_3 M^2 + \alpha_4 m M + \alpha_5 m^2, \\ \frac{dm}{dt} &= -\alpha_0 - \alpha_1 M - \alpha_2 m - \alpha_3 M^2 - \alpha_4 m M - \alpha_5 m^2. \end{aligned} \quad (8)$$

Corresponding to reactions up to the second order, terms on the right-hand side are also kept up to the second-order polynomials of M and m . Due to the laws of mass conservation, i.e. $M(t) + m(t) = m_{tot}$, five free parameters in the second equation of m can be completely fixed. It is further noted that since in the current case at least seven concentrations of

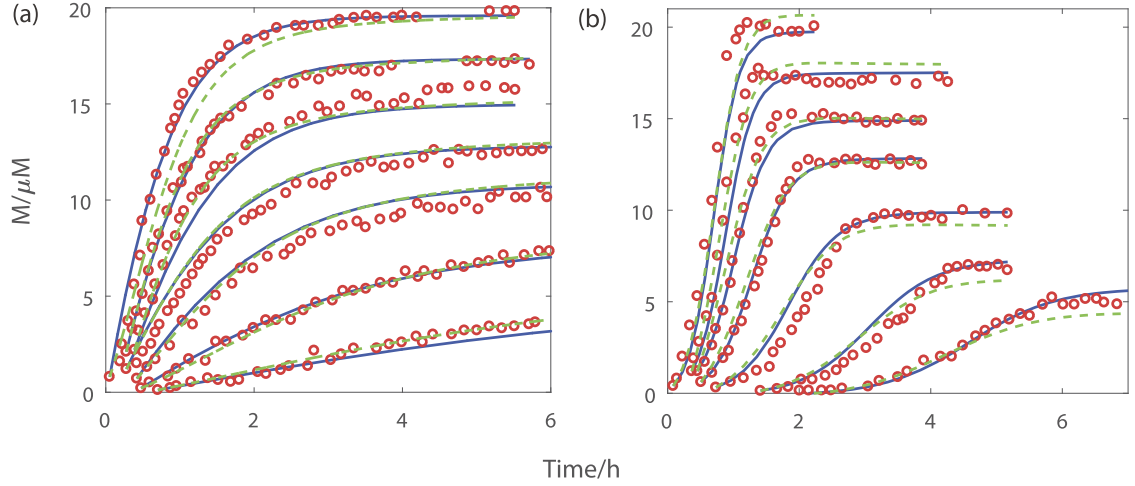


Fig. 6. Kinetics of actin aggregation in (a) KCl and (b) MgCl_2 solutions respectively. Red circles stand for experimental data in [44], blue solid lines for predictions of the data-driven model in (8), green dashed lines for the physical-based model in (9).

Table 4

Learned coefficients for data-driven model of actin aggregation.

Filaments	Parameters					
	α_0	α_1	α_2	α_3	α_4	α_5
actin in KCl	4.62×10^{-1}	-2.16×10^{-1}	-5.49×10^{-1}	5.70×10^{-3}	1.10×10^{-1}	7.87×10^1
actin in MgCl_2	0	-1.41×10^{-2}	9.20×10^{-3}	-3.75×10^{-2}	2.28×10^{-1}	3.10×10^1

actin are considered, a global fitting of data with different m_{tot} at the same time is essential for the learning procedure of ODENet.

Beyond the good agreement between ODENet predictions and experimental data as shown in Fig. 6, the learned ODE parameters for actin aggregation given in Table 4 are worthy of further clarification, especially their physical meanings. Terms α_0 , $\alpha_2 m$ and $\alpha_5 m^2$ together account for primary nucleation within two monomers. $\alpha_1 m$ represents the process of degradation (or monomer dissociation). Since it makes a negative contribution to the filament concentration, α_1 is always negative as expected. The term $\alpha_4 m M$ comes from actin filament elongation, which depends on not only the monomer concentration but also the filament concentration. Only the physical meaning of $\alpha_3 M^2$ is not so straightforward, which may originate from some complicated interactions between filaments, like annealing or clumping. However, based on the coefficients listed in Table 4, we cannot tell the difference between actin incubating with KCl and with MgCl_2 . These limitations motivate us to consider a more physical-based model.

Physical based model: According to the general theory for actin aggregation [45], besides the mass concentrations of actin filaments and monomers, the number concentration of actin filaments P also plays a non-negligible role in constructing a complete description of actin growth. So instead of two ODEs, in principle we should consider three coupled equations as the “correct” model. However, as the experimental data contain no direct information on P , the variable P is actually a hidden one. If we do not write it out explicitly, there is no way to learn it in a purely data-driven modeling. Up to the second-order polynomials, we have

$$\begin{aligned}
 \frac{dP}{dt} &= \alpha_0 + \alpha_1 m + \alpha_2 m^2 + \alpha_3 m M + \alpha_4 P + \alpha_5 M + \alpha_6 P^2 + \alpha_7 P M, \\
 \frac{dM}{dt} &= \alpha_0 + \alpha_1 m + 2\alpha_2 m^2 + \alpha_3 m M + \alpha_8 M + \alpha_9 m P + \alpha_{10} P, \\
 \frac{dm}{dt} &= -\alpha_0 - \alpha_1 m - 2\alpha_2 m^2 - \alpha_3 m M - \alpha_8 M - \alpha_9 m P - \alpha_{10} P.
 \end{aligned} \tag{9}$$

Here those terms without any physical meaning have been removed, and only eleven free coefficients are kept instead of thirty. The remaining terms all have clear physical interpretations. To be exact, terms α_0 , $\alpha_1 m$ and $\alpha_2 m^2$ stand for primary nucleation, $\alpha_3 m M$ for secondary nucleation, $\alpha_4 P$ and $\alpha_8 M$ for fibril degradation, $\alpha_5 M$ for fragmentation, $\alpha_6 P^2$ for annealing, $\alpha_7 P M$ for clumping, $\alpha_9 m P$ for elongation and $\alpha_{10} P$ for monomer dissociation respectively.

The hidden variable P plays a key role in the physical-based model. But how to learn it is a highly non-trivial task. Since we cannot make a direct comparison between the predicted $P(t)$ with its true values, it will not appear in the loss

Table 5

Learned coefficients for physical-based model of actin aggregation. Unmentioned coefficients are all zeros.

Filaments	Parameters					
	α_1	α_2	α_3	α_4	α_9	α_{10}
actin in KCl	-5.12×10^{-2}	7.98×10^{-3}	1.16×10^{-2}	-5.33×10^{-1}	7.39×10^{-1}	-8.82×10^{-1}
actin in MgCl_2	2.15×10^{-2}	0	2.3×10^{-2}	0	1.19×10^1	-2.97×10^1

function. But as $M(t)$ depends on $P(t)$ according to (9), optimizing the predictions on $M(t)$ will also lead to an optimization of predicted $P(t)$ simultaneously, as long as the solution of the problem is a fixed-point (it is a general belief, though we are unable to prove its convergence). To help the convergence, the initial values of $P(t)$ were estimated through the approximation $dM/dt \approx \alpha_9 mP$, as we regard elongation as the dominant process. Without a proper guess on the initial values, the learning process will fail with a very high probability.

During the learning procedure of ODENet, terms α_0 , $\alpha_5 M$, $\alpha_6 P^2$, $\alpha_7 PM$, and $\alpha_8 M$ are eliminated by sparsity requirement, indicating the corresponding processes may not be essential for modeling. The remaining terms listed in Table 5 suggest a clear molecular mechanism for the actin aggregation, including primary nucleation (indicated by $\alpha_1 m$ and $\alpha_2 m^2$), elongation ($\alpha_9 mP$), surface catalyzed secondary nucleation ($\alpha_3 mM$), monomer dissociation ($\alpha_{10} P$) and degradation $\alpha_4 P$. Among them, the first three processes are dominant for microfilament growth, while the latter two are responsible for maintaining the equilibrium state. Further comparing the model coefficients learned from ODENet, the elongation rate for actin aggregation in KCl solution is much smaller than in MgCl_2 solution, suggesting the former process is dominated by primary nucleation, while the latter is dominated by elongation and secondary nucleation instead. This dramatic distinction is believed to be caused by different chemical valences of K^+ and Mg^{2+} . Therefore, the physical-based modeling by ODENet indeed provides new insights into those unknown phenomena we are interested in. Qualitatively, the physical-based model uncovered by ODENet is in good agreement with previous models constructed purely by the human brain [45]. In the latter, primary nucleation, elongation, and fragmentation (a kind of secondary nucleation) were considered as three dominant processes for actin aggregation, and the difference between KCl and MgCl_2 solutions lies in the strength of primary nucleation v.s. secondary nucleation.

Box 3: Data-driven v.s. physical-based modeling of actin aggregation

Goal: Compare the purely data-driven model with the physical-based model on kinetics of actin aggregation.

Data: Mass concentration $M(t)$ of actin filaments recorded at different time points in ThT fluorescence experiments. Seven protein concentrations and two buffer conditions are taken into consideration.

Setup: There are two separate setups for the learning procedure:

1. **Purely data-driven model.** Without any pre-knowledge of the model, we just need single equation of mass concentration $M(t)$ to learn the dynamics. To account for the concentration dependence, an additional variable $m(t) = m_{\text{tot}} - M(t)$ is introduced too. Mimicking the function on the right-hand side of ODEs by polynomials up to the second-order, we have to optimize six coefficients $\theta = (\theta_{ij})_{2 \times 6}$, where $\theta_{2j} = -\theta_{1j}$, $j = 1, 2, \dots, 6$.
2. **Physical-based model.** According to the general theory for actin aggregation [45], another hidden variable – the number concentration of actin filaments $P(t)$ is introduced into the model. Furthermore, we require all kept terms to have a clear physical meaning to account for all possible mechanisms for actin growth. In this case, we have three ordinary differential equations with eleven undetermined coefficients. The hidden variable $P(t)$ is constructed in a self-iterative way from the approximation $dM/dt \approx \alpha_9 mP$ with a pre-knowledge that elongation makes a major contribution to the mass growth of actin filaments.

Learning: Parameters θ are optimized according to the procedure of Algorithm 1. Sparsity requirement is taken.

Results: Two simple models with and without hidden physical variable $P(t)$ are learned separately, both of which can fit ThT trajectories quite well.

5. Conclusion and discussion

In this work, we proposed a general and flexible network called ODENet for revealing hidden ODE dynamics from time-series data. A significant difference of ODENet from the state-of-art regression-based methods like SINDy is the adoption of integration of explicit ODEs along the time trajectory. By further combining with classical machine learning skills, like data batching, back-propagation and optimization, ODENet inherits the advantages of both machine learning and ODEs. On one hand, the embedding of ODEs makes the whole procedure transparent and interpretable. On the other hand, the schemes of machine learning enable data handling, paralleling, and optimization to be easily and efficiently implemented.

As illustrated through several novel examples including Lotka-Volterra models for population dynamics, strange attractors of Lorenz equations, and the kinetics of actin aggregation into microfilaments, ODENet shows great merits in several aspects: (1) the ability to deal with data not equally spaced, of a high noise to signal ratio, etc.; (2) tolerance with large sampling time steps; (3) explicitly deriving interpretable models with fewer parameters; (4) efficiently optimizing parameters by BP algorithms; (5) very flexible network structure ready for the incorporation of various new approaches. Therefore, we expect wider applications of ODENet in various branches of natural science, as well as non-trivial extensions to stochastic ODEs and PDEs for a better description of the real world in the near future.

CRediT authorship contribution statement

Pipi Hu: Investigation, Methodology, Software, Validation, Writing – original draft. **Wuyue Yang:** Data curation, Investigation, Software, Validation, Writing – original draft. **Yi Zhu:** Conceptualization, Investigation, Methodology, Writing – review & editing. **Liu Hong:** Conceptualization, Investigation, Methodology, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by the National Key R&D Program of China (Grant No. 2021YFA0719200), the National Natural Science Foundation of China (Grant Nos. 21877070 and 11871299), and the Hundred-Talent Program of Sun Yat-Sen University.

References

- [1] Todd D. Little, *The Oxford Handbook of Quantitative Methods in Psychology: vol. 2: Statistical Analysis*, vol. 2, Oxford University Press, 2013.
- [2] Jeffrey L. Elman, Finding structure in time, *Cogn. Sci.* 14 (2) (1990) 179–211.
- [3] Sepp Hochreiter, Jürgen Schmidhuber, Long short-term memory, *Neural Comput.* 9 (8) (1997) 1735–1780.
- [4] Ricky T.Q. Chen, Yulia Rubanova, Jesse Bettencourt, David K. Duvenaud, Neural ordinary differential equations, in: *Advances in Neural Information Processing Systems*, 2018, pp. 6571–6583.
- [5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun, Deep residual learning for image recognition, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [6] Weinan E, A proposal on machine learning via dynamical systems, *Commun. Math. Stat.* 5 (1) (2017) 1–11.
- [7] Weinan E, Chao Ma, Lei Wu, Machine learning from a continuous viewpoint, I, *Sci. China Math.* 63 (11) (2020) 2233–2266.
- [8] Maziar Raissi, Paris Perdikaris, George Em Karniadakis, Multistep neural networks for data-driven discovery of nonlinear dynamical systems, *arXiv preprint arXiv:1801.01236*, 2018.
- [9] Josh Bongard, Hod Lipson, Automated reverse engineering of nonlinear dynamical systems, *Proc. Natl. Acad. Sci.* 104 (24) (2007) 9943–9948.
- [10] Michael Schmidt, Hod Lipson, Distilling free-form natural laws from experimental data, *Science* 324 (5923) (2009) 81–85.
- [11] John R. Koza, John R. Koza, *Genetic Programming: on the Programming of Computers by Means of Natural Selection*, vol. 1, MIT Press, 1992.
- [12] Steven L. Brunton, Joshua L. Proctor, J. Nathan Kutz, Discovering governing equations from data by sparse identification of nonlinear dynamical systems, *Proc. Natl. Acad. Sci. U.S.A.* 113 (15) (2016) 3932–3937.
- [13] Samuel H. Rudy, Steven L. Brunton, Joshua L. Proctor, J. Nathan Kutz, Data-driven discovery of partial differential equations, *Sci. Adv.* 3 (4) (2017) e1602614.
- [14] Samuel H. Rudy, J. Nathan Kutz, Steven L. Brunton, Deep learning of dynamics and signal-noise decomposition with time-stepping constraints, *J. Comput. Phys.* 396 (2019) 483–506.
- [15] Zichao Long, Yiping Lu, Xianzhong Ma, Bin Dong, Pde-net: learning pdes from data, in: *International Conference on Machine Learning*, 2018, pp. 3208–3216.
- [16] Zichao Long, Yiping Lu, Bin Dong, Pde-net 2.0: learning pdes from data with a numeric-symbolic hybrid deep network, *J. Comput. Phys.* 399 (2019) 108925.
- [17] Maziar Raissi, Paris Perdikaris, George E. Karniadakis, Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *J. Comput. Phys.* 378 (2019) 686–707.
- [18] Maziar Raissi, Deep hidden physics models: deep learning of nonlinear partial differential equations, *J. Mach. Learn. Res.* 19 (1) (2018) 932–955.
- [19] Gert-Jan Both, Subham Choudhury, Pierre Sens, Remy Kusters, Deepmod: deep learning for model discovery in noisy data, *J. Comput. Phys.* 428 (2021) 109985.
- [20] Zhao Chen, Yang Liu, Hao Sun, Physics-informed learning of governing equations from scarce data, *Nat. Commun.* 12 (1) (2021) 1–13.
- [21] Guochang Lin, Pipi Hu, Fukai Chen, Xiang Chen, Junqing Chen, Jun Wang, Zuoqiang Shi, Binet: learning to solve partial differential equations with boundary integral networks, *arXiv preprint arXiv:2110.00352*, 2021.
- [22] Qin Lou, Xuhui Meng, George Em Karniadakis, Physics-informed neural networks for solving forward and inverse flow problems via the Boltzmann-BGK formulation, *J. Comput. Phys.* 447 (2021) 110676.
- [23] Lu Lu, Pengzhan Jin, George Em Karniadakis, Deeponet: learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators, *arXiv preprint arXiv:1910.03193*, 2019.
- [24] Yang Li, Jinqiao Duan, A data-driven approach for discovering stochastic dynamical systems with non-Gaussian Lévy noise, *Phys. D: Nonlinear Phenom.* 417 (2021) 132830.
- [25] Tom Goldstein, Stanley Osher, The split Bregman method for l1-regularized problems, *SIAM J. Imaging Sci.* 2 (2) (2009) 323–343.
- [26] Stephen J. Wright, Coordinate descent algorithms, *Math. Program.* 151 (1) (2015) 3–34.
- [27] Mark Schmidt, Nicolas L. Roux, Francis R. Bach, Convergence rates of inexact proximal-gradient methods for convex optimization, in: *Advances in Neural Information Processing Systems*, 2011, pp. 1458–1466.

- [28] Diederik P. Kingma, Jimmy Ba, Adam: a method for stochastic optimization, arXiv preprint arXiv:1412.6980, 2014.
- [29] Yann LeCun, Bernhard Boser, John S. Denker, Donnie Henderson, Richard E. Howard, Wayne Hubbard, Lawrence D. Jackel, Backpropagation applied to handwritten zip code recognition, *Neural Comput.* 1 (4) (1989) 541–551.
- [30] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, Adam Lerer, Automatic Differentiation in Pytorch, 2017.
- [31] M. Calvo, J.I. Montijano, L. Rande, A fifth-order interpolant for the Dormand and Prince Runge-Kutta method, *J. Comput. Appl. Math.* 29 (1) (1990) 91–100.
- [32] Ernst Hairer, Gerhard Wanner, Stiff differential equations solved by Radau methods, *J. Comput. Appl. Math.* 111 (1–2) (1999) 93–111.
- [33] John R. Dormand, Peter J. Prince, A family of embedded Runge-Kutta formulae, *J. Comput. Appl. Math.* 6 (1) (1980) 19–26.
- [34] Jiequn Han, Chao Ma, Zheng Ma, E. Weinan, Uniformly accurate machine learning-based hydrodynamic models for kinetic equations, *Proc. Natl. Acad. Sci.* 116 (44) (2019) 21983–21991.
- [35] Wuyue Yang, Liangrong Peng, Yi Zhu, Liu Hong, When machine learning meets multiscale modeling in chemical reactions, *J. Chem. Phys.* 153 (9) (2020) 094117.
- [36] Alfred J. Lotka, Elements of physical biology, *Sci. Prog. Twent. Century* 21 (82) (1926) 341–343.
- [37] Vito Volterra, Variazioni e fluttuazioni del numero d'individui in specie animali conviventi, C. Ferrari, 1927.
- [38] Paul A. Samuelson, Generalized predator-prey oscillations in ecological and economic equilibrium, *Proc. Natl. Acad. Sci.* 68 (5) (1971) 980–983.
- [39] Bi-Huei Tsai, Chih-Jen Chang, Chun-Hsien Chang, Elucidating the consumption and co2 emissions of fossil fuels and low-carbon energy in the United States using Lotka–Volterra models, *Energy* 100 (2016) 416–424.
- [40] Robert D. Holt, John Pickering, Infectious disease and species coexistence: a model of Lotka–Volterra form, *Am. Nat.* 126 (2) (1985) 196–211.
- [41] Robert M. May, Limit cycles in predator-prey communities, *Science* 177 (4052) (1972) 900–902.
- [42] Edward N. Lorenz, Deterministic nonperiodic flow, *J. Atmos. Sci.* 20 (2) (1963) 130–141.
- [43] Fumio Oosawa, Michiki Kasai, A theory of linear and helical aggregations of macromolecules, *J. Mol. Biol.* 4 (1) (1962) 10–21.
- [44] Albrecht Wegner, Paula Savko, Fragmentation of actin filaments, *Biochemistry* 21 (8) (1982) 1909–1913.
- [45] Liu Hong, Chiu Fan Lee, Ya Jing Huang, Statistical Mechanics and Kinetics of Amyloid Fibrillation, 9 2017.